

GUIDE DÉPLOIEMENT ET GESTION D'UN SERVEUR GIT LOCAL

Installation, Configuration et Gestion d'un Serveur Git

Sommaire

1. Installation de Git sur le Serveur
2. Création et Configuration des Comptes Développeurs
3. Gestion de la Sécurité (Clés SSH)
4. Création de Dépôts Centraux Bare
5. Migration et Connexion de Projets Existants
6. Flux de Travail Développeur (Clone, Branches & Intégration VS Code)
7. Commandes de Diagnostic et Visualisation
8. Résolution des Erreurs Courantes
9. Maintenance (Modification et Suppression)

1. Installation de Git sur le Serveur

Pour préparer un serveur local dédié (basé sur Debian/Ubuntu), connectez-vous d'abord en SSH avec un compte administratif, puis exécutez la mise à jour des paquets et l'installation :

```
# Mise à jour de la liste des paquets et du système (Recommandé)
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Installation du paquet Git
```

```
sudo apt install git -y
```

```
# Vérification du succès de l'installation
```

```
git --version
```

2. Création et Configuration des Comptes Développeurs

Pour que les développeurs puissent s'authentifier sur le serveur via SSH, créez un compte utilisateur système Linux pour chacun d'eux.

Création d'un utilisateur

```
sudo adduser nom_developpeur
```

Suivez les invites pour définir un mot de passe fort et renseigner les détails de l'utilisateur.

Ajouter le nom complet (Full Name) après création

```
sudo chfn -f "Prénom Nom" nom_developpeur
```

Vérification de la liste des utilisateurs du serveur

Pour s'assurer que les utilisateurs sont bien enregistrés dans le système :

```
cat /etc/passwd
```

```
# Ou alternativement :
```

```
getent passwd
```

Règle de Configuration d'Identité Git :

L'administrateur n'a pas besoin d'enregistrer ses identifiants Git globaux sur le serveur car le serveur n'effectue pas de commits directement. **Chaque développeur doit configurer son identité sur sa propre machine locale** afin que l'historique des commits attribue correctement les modifications :

```
git config --global user.name "Prénom Nom"
```

```
git config --global user.email "developpeur@company.com"
```

3. Gestion de la Sécurité (Clés SSH)

L'utilisation de clés SSH est indispensable pour sécuriser les échanges et éviter de saisir le mot de passe utilisateur à chaque interaction Git.

- **Authentification sans mot de passe** : Utilise un mécanisme de clés asymétriques plus robuste contre les attaques de force brute.
- **Chiffrement des flux** : Sécurise l'intégralité du code transféré entre la machine locale et le serveur.
- **Contrôle d'accès** : Permet d'autoriser ou révoquer finement l'accès d'un développeur.

Déploiement d'une clé publique sur le serveur :

Le développeur génère sa paire de clés localement (ssh-keygen) et transmet sa clé publique (id_rsa.pub) à l'administrateur, qui l'ajoute au compte du développeur sur le serveur :

```
# Exécuté sur le serveur pour injecter la clé reçue
```

```
cat chemin_vers_cle_publique.pub >>
```

```
/home/nom_developpeur/.ssh/authorized_keys
```

```
# Ajustement des permissions impératives pour SSH
```

```
sudo chmod 700 /home/nom_developpeur/.ssh
```

```
sudo chmod 600 /home/nom_developpeur/.ssh/authorized_keys
```

```
sudo chown -R nom_developpeur:nom_developpeur
```

```
/home/nom_developpeur/.ssh
```

4. Création de Dépôts Centraux Bare

Sur le serveur Git, les projets sont stockés sous forme de dépôts dits **"Bare"**. Un dépôt bare ne contient pas d'espace de travail directement éditable (pas de fichiers sources visibles), mais uniquement la base de données Git (historique, branches, objets). Par convention, le dossier se termine par l'extension .git.

```
# Création du dossier du dépôt (Exemple dans l'espace web ou partagé)
```

```
sudo mkdir -p /var/www/html/mon_projet.git
```

```
# Initialisation en mode bare
```

```
cd /var/www/html/mon_projet.git
```

```
sudo git init --bare
```

Configuration de la branche par défaut (Optionnel)

Pour éviter les avertissements et harmoniser le nom de la branche initiale (ex: main au lieu de master) :

```
sudo git config --global init.defaultBranch main
```

5. Migration et Connexion de Projets Existants

Si vous possédez un projet existant sur un autre serveur ou sur une machine de développement et que vous souhaitez le centralisée sur le nouveau serveur Git local, suivez scrupuleusement ces étapes **depuis le répertoire du projet source** :

Étape 5.1 : Initialisation locale (si ce n'est pas déjà un dépôt Git)

```
cd /chemin/vers/votre/projet_existant  
git init
```

Étape 5.2 : Nettoyage et configuration du pointeur distant (Remote)

Si le projet pointe déjà vers une autre plateforme (comme GitHub) et que vous souhaitez basculer vers votre serveur local :

```
# Vérifier les liaisons distantes existantes  
git remote -v
```

```
# Si 'origin' existe déjà et pointe vers le mauvais serveur, supprimez-le :  
git remote remove origin
```

```
# Ajouter la liaison vers votre nouveau serveur Git local  
git remote add origin  
nom_developpeur@IP_SERVEUR:/var/www/html/mon_projet.git
```

Étape 5.3 : Préparation et premier envoi des fichiers

```
# Indexer l'intégralité du projet  
git add .
```

```
# Valider l'état initial localement  
git commit -m "Import initial du projet existant"
```

```
# Pousser l'ensemble des branches vers le serveur local  
git push -u origin --all
```

6. Flux de Travail Développeur (Clone, Branches & Intégration VS Code)

6.1 Récupération d'un projet : Le "git clone"

Lorsqu'un développeur rejoint le projet ou s'installe sur une nouvelle machine de travail, il télécharge l'intégralité du dépôt central vers son ordinateur local à l'aide de la commande git clone :

```
git clone  
ssh://nom_developpeur@IP_SERVEUR:/var/www/html/mon_projet.git
```

Cette commande crée automatiquement un dossier local nommé selon le projet, initialise Git et configure la liaison distante origin.

6.2 Travailler avec Visual Studio Code (VS Code)

1. Ouvrez le terminal de votre machine de travail et positionnez-vous dans le dossier qui vient d'être cloné : `cd mon_projet`.
2. Lancez l'éditeur directement dans ce dossier en tapant : `code ..`
3. Utilisez l'onglet **Source Control** (raccourci Ctrl+Shift+G) dans la barre latérale gauche pour suivre visuellement vos modifications, effectuer des *Staging* (équivalent visuel du git add), rédiger vos messages de commit et effectuer vos synchronisations (Push / Pull) sans ouvrir le terminal.

6.3 Comprendre et manipuler les branches simplement

Une branche est une ligne de développement parallèle. La branche principale (main ou master) contient le code stable de production. Créer une branche permet d'isoler le développement d'une fonctionnalité ou la correction d'un bug sans perturber le

travail des autres.

Objectif Action	Commande Git Associée
Créer une branche et basculer dessus immédiatement	git checkout -b nom_nouvelle_branche
Basculer vers une branche existante	git checkout nom_branche
Fusionner les travaux d'une branche de test dans la branche courante	git merge nom_branche_a_fusionner
Pousser une branche spécifique sur le serveur distant	git push origin nom_branche

7. Commandes de Diagnostic et Visualisation

Ces commandes permettent de connaître précisément l'état de vos fichiers et l'historique des branches :

- **git status** : Affiche l'état du répertoire de travail (fichiers modifiés, non suivis ou indexés).
- **git log** : Affiche l'historique chronologique des commits effectués sur la branche active.
- **git log nom_branche** : Permet de consulter les commits d'une branche distante ou parallèle sans y basculer.
- **git branch -a** : Liste l'intégralité des branches locales ET distantes synchronisées.
- **git branch -r** : Liste exclusivement les branches présentes sur le serveur distant.
- **git diff branche1..branche2** : Affiche les différences de code ligne par ligne

entre deux branches distinctes.

- **git show nom_branche:chemin/fichier** : Permet de lire le contenu d'un fichier tel qu'il existe sur une autre branche sans changer d'espace de travail.

8. Résolution des Erreurs Courantes

Erreur 1 : fatal: not a git repository (or any of the parent directories): .git

- **Cause** : Vous exécutez une commande Git dans un répertoire système standard qui n'a pas été initialisé avec Git.
- **Solution** : Vérifiez votre positionnement avec `pwd`, déplacez-vous dans le dossier de votre projet avec `cd`, ou initialisez le dossier en faisant `git init`.

Erreur 2 : error: remote unpack failed: unable to create temporary object directory

- **Cause** : Lors du `git push`, le serveur refuse la création de fichiers temporaires nécessaires à l'injection du code. Cela provient généralement d'un problème de droits d'accès sur le dossier du serveur ou d'un disque saturé.
- **Solution Permissions** : Réappliquez les droits au propriétaire légitime ou au groupe de développement sur le serveur :

```
sudo chown -R nom_developpeur:nom_groupe /var/www/html/mon_projet.git  
sudo chmod -R 775 /var/www/html/mon_projet.git
```
- **Solution Espace Disque** : Vérifiez l'espace disponible sur le serveur distant avec la commande : `df -h`.

Erreur 3 : fatal: detected dubious ownership in repository at...

- **Cause** : Git applique des règles strictes de sécurité si l'utilisateur qui exécute la commande est différent du propriétaire initial du dossier.
- **Solution** : Déclarez le répertoire comme de confiance de manière globale :

```
git config --global --add safe.directory  
/var/www/html/nom_dossier_concerne
```

Erreur 4 : error: src refspec master does not match any

- **Cause** : Vous tentez de pousser une branche (ex: master) qui n'existe pas encore localement ou qui ne contient aucun commit initialisé.
- **Solution** : Créez au moins un commit initial (git add . puis git commit -m "init") avant de pousser, ou validez le nom exact de votre branche locale active via git branch.

9. Maintenance (Modification et Suppression)

Modifier le nom d'un utilisateur système

```
sudo usermod -l nouveau_nom ancien_nom  
# Optionnel : Renommer également son dossier Home personnel  
sudo usermod -d /home/nouveau_nom -m nouveau_nom
```

Supprimer un utilisateur système

```
sudo deluser --remove-home nom_utilisateur
```

Supprimer définitivement un dépôt Git du serveur

Pour purger un projet et l'intégralité de son historique sur le serveur :

```
sudo rm -rf /var/www/html/mon_projet.git
```

Attention : Cette action est irréversible et supprime définitivement toutes les données du dépôt centralisé.